

Retail Sales & Returns Analytics

A Microsoft Fabric Data Engineering Project

Azure source files | Fabric Data Factory | OneLake Lakehouse | PySpark | Power BI

Prepared for	Nandini Babu
Report date	1 October 2026
Evidence supplied	14 screenshots across two PDFs
Main deliverable	A product-level KPI table and a Retail Sales Overview report

Project purpose

Bring orders, inventory and returns into a common analytical environment, improve the usability of inconsistent source data, and present product-level operational indicators in Power BI.

How this report uses the evidence

The supplied screenshots are reproduced as project evidence. Visible names, counts and outputs are described directly. Transformation rules, metric definitions and production improvements that cannot be confirmed from the screenshots are explicitly identified as proposed or to be verified. No notebook code, execution logs or original source datasets were supplied.

This report documents the Fabric project shown in these attachments. It does not combine it with the earlier Azure Databricks and Unity Catalog project.

1. Executive summary and scope

The project brings three retail datasets from an Azure storage container into a Fabric lakehouse named **lake1**, in the **Retail trial** workspace. A pipeline named **Pipeline_1** contains copy activities for orders, returns and inventory. Bronze files are stored as Parquet. A PySpark notebook named **Notebook_2** displays raw data and an enriched result. The lakehouse explorer shows Silver tables and the Gold table **gold_product_kpis**.

A semantic model named **retail** exposes the Gold table to a report titled **Retail Overview**. The final screenshot contains a Total Orders card, stock and return-amount bar charts, an orders/returns donut, an orders-by-product pie chart, a table and a product filter.

Business questions

Which products have the most stock? Which products contribute the highest recorded return amounts? How many orders and returns appear in the current filter context? How does the distribution of orders vary across product names? Which fields need additional validation before the results can support business decisions?

Observed outcomes

Evidence	Visible result
Raw inventory preview	7 columns and 15 rows
Enriched notebook output	Silver orders: 13; Enriched rows: 13
Final report card	13 total orders
Final report table totals	13 orders; 13 returns; 239 stock
Percentage total in screenshot	1,300.00; requires correction

These are observations from screenshot snapshots. They are not independently reconciled totals from source files. In particular, the 13-row enriched output does not by itself establish the number of Gold product groups or unique customers.

2. Architecture and responsibilities

End-to-end sequence

Stage	Project object	Purpose
Source	Azure container: source	orders_data.csv, inventory_data.json, returns_data.xlsx
Ingestion	Pipeline_1	Copy the three datasets into the lakehouse
Bronze	lake1 / Files / Bronze	Retain raw data in Parquet format
Silver	silver_orders, silver_returns, silver_inventory	Organize cleaned, typed datasets
Gold	gold_product_kpis	Expose product-level analytical fields
Semantic model	retail	Provide report-facing fields and calculations
Reporting	Retail Overview	Present charts, card, table and product filtering

The design follows a medallion pattern: Bronze holds raw data, Silver improves data quality and consistency, and Gold provides curated analytical outputs. Microsoft documents these layers as a Fabric lakehouse architecture pattern [1]. The screenshot shows the layers together within one lakehouse rather than proving that separate lakehouses were created for each layer.

Why the layers matter

Raw preservation makes it possible to revisit transformation decisions. Separate Silver tables prevent inconsistent source formats from spreading into every chart. A Gold product table gives the report a simpler entry point. The semantic model can then centralize business calculations so totals behave consistently across visuals.

Storage distinction

The source screenshot shows an Azure storage container. The attachments do not prove whether hierarchical namespace is enabled, so the source is described as Azure storage rather than definitively as ADLS Gen2. The Fabric destination is the OneLake-backed lakehouse. No REST API, HTTP source, Azure SQL source or Databricks workspace is evidenced in this set.

3. Source files in Azure storage

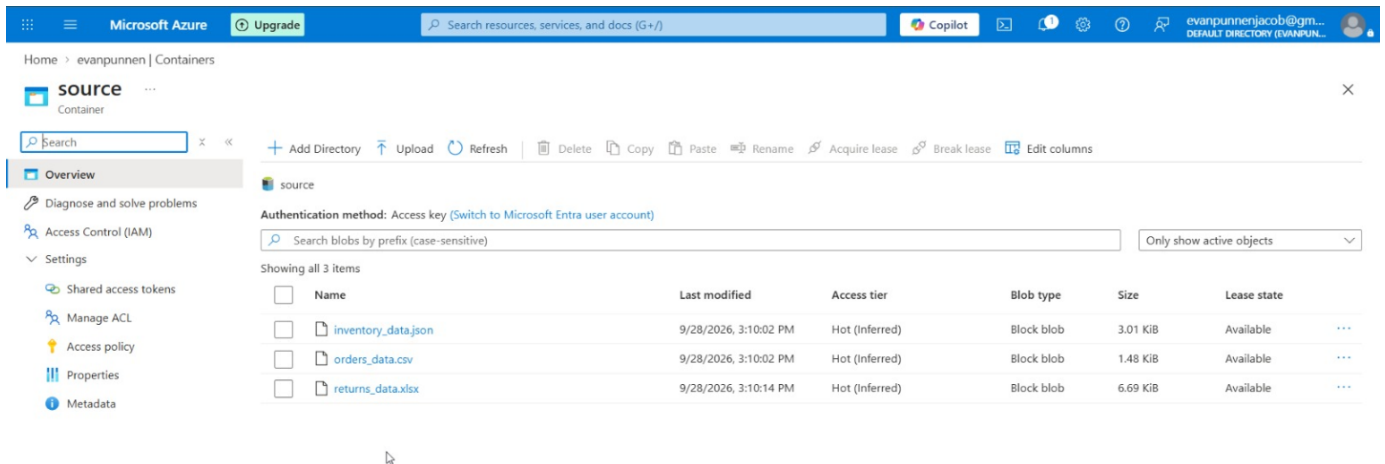


Figure 1. Azure source container with three uploaded files. Source: Part 2 PDF, page 4.

What was prepared

The container named **source** contains **inventory_data.json**, **orders_data.csv** and **returns_data.xlsx**. These represent product availability, customer orders and return transactions. The mixture of JSON, CSV and Excel provides a practical example of integrating different file formats.

Why this stage matters

Centralizing the source files gives the ingestion pipeline a clear location to read from. Source file identity and format must be preserved so that the correct parser and column mappings are applied to each dataset.

Evidence boundary

The container contents are visible, but connection configuration, permissions and exact ingestion settings are not. The screenshot shows an access-key authentication selection; it does not expose the key value or establish how Pipeline_1 authenticated.

4. Ingestion with Pipeline_1

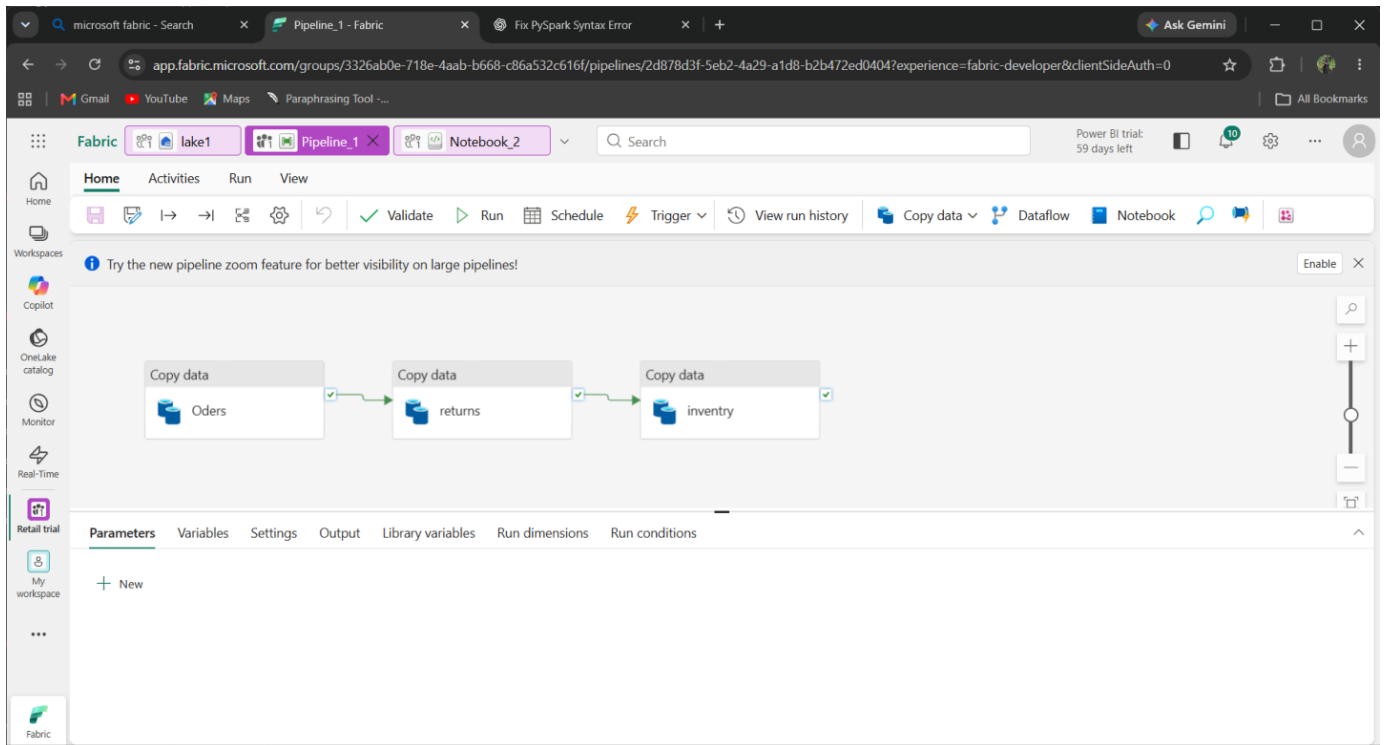


Figure 2. Three connected Fabric copy activities. Source: first PDF, page 3.

Pipeline design

The copy activities are labeled **Orders**, **returns** and **inventory** in the screenshot. They are linked in sequence by green success-dependency connectors. This means the following activity is configured to proceed after the preceding activity succeeds. The spelling inventory is retained here as the displayed activity name.

Functional explanation

Each activity is responsible for moving one input dataset into the lakehouse. Fabric Copy activity supports data movement and source-to-destination mapping [2]. The subsequent Bronze screenshot provides evidence that destination files exist.

What is not demonstrated

The pipeline canvas is not a complete run log. The screenshot does not prove a schedule, retry policy, alert, incremental watermark or notebook orchestration. Those would need separate configuration and execution evidence.

5. Bronze landing layer

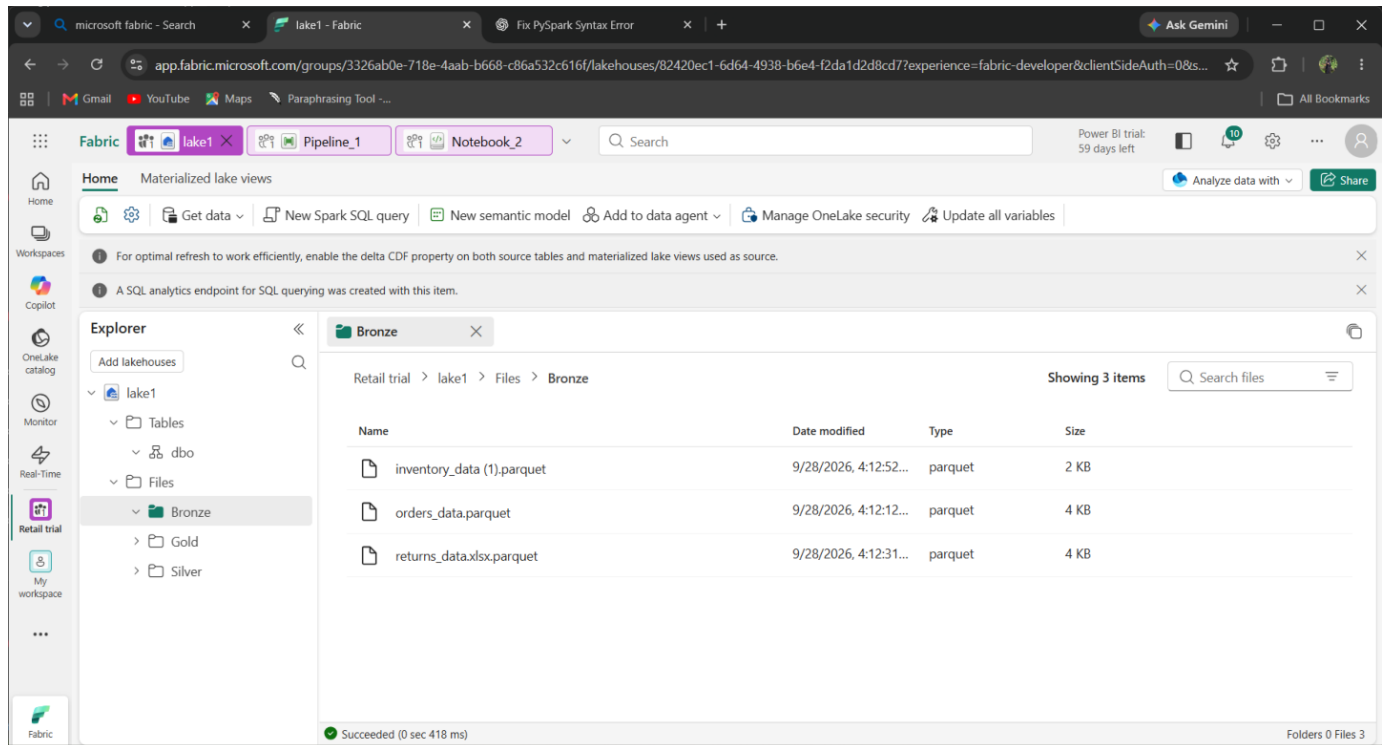


Figure 3. Parquet files in lake1 / Files / Bronze. Source: first PDF, page 2.

Observed files

The Bronze folder contains **inventory_data (1).parquet**, **orders_data.parquet** and **returns_data.xlsx.parquet**. The lakehouse also shows Gold and Silver folders. Although the returns filename contains `xlsx`, the destination file type is Parquet.

Purpose of Bronze

This layer is the initial landing area. Converting a file to Parquet changes its storage representation; it does not automatically repair invalid values, inconsistent dates or missing fields. Keeping the raw values available supports debugging and reconciliation.

Operational improvement

Use consistent destination names and capture input filename, ingestion time and batch identifier. These additions are proposed improvements, not fields demonstrated in the screenshots.

6. Notebook access and inventory preview

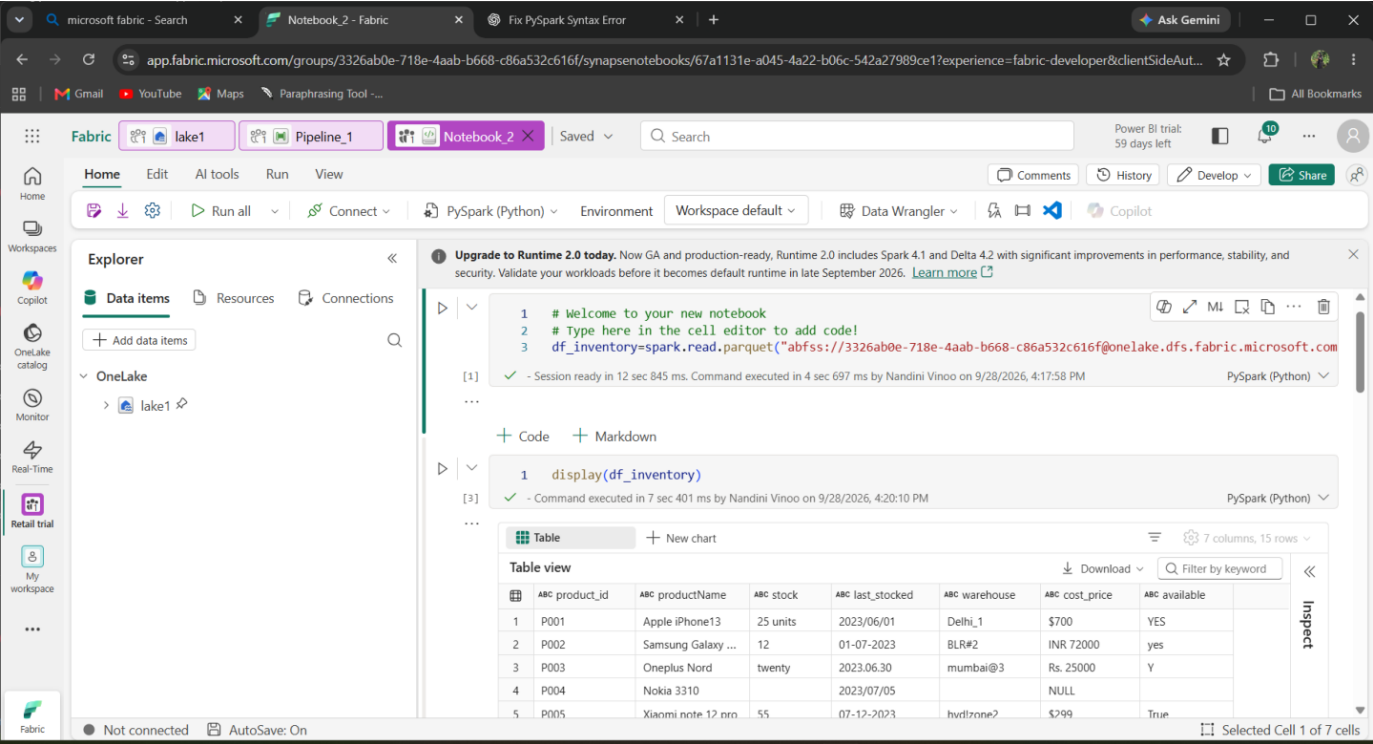


Figure 4. Notebook_2 reads a Bronze Parquet path and displays inventory. Source: first PDF, page 4.

Observed notebook operation

A cell assigns a Parquet read to **df_inventory**, using an abfss path to OneLake. Another cell calls **display(df_inventory)**. The preview reports **7 columns and 15 rows**. Green execution indicators are visible for the displayed cells.

Inventory fields

The visible fields are product_id, productName, stock, last_stocked, warehouse, cost_price and available. Stock includes values such as 25 units and twenty; cost values include currency symbols or text; availability uses several spellings.

Interpretation

The preview is useful for profiling before transformations. Stock should be suitable for numeric comparison, dates for date operations, and availability for a controlled category. The full cleaning code is not visible, so exact parsing rules cannot be reconstructed from this screenshot.

7. Inventory quality and variable verification

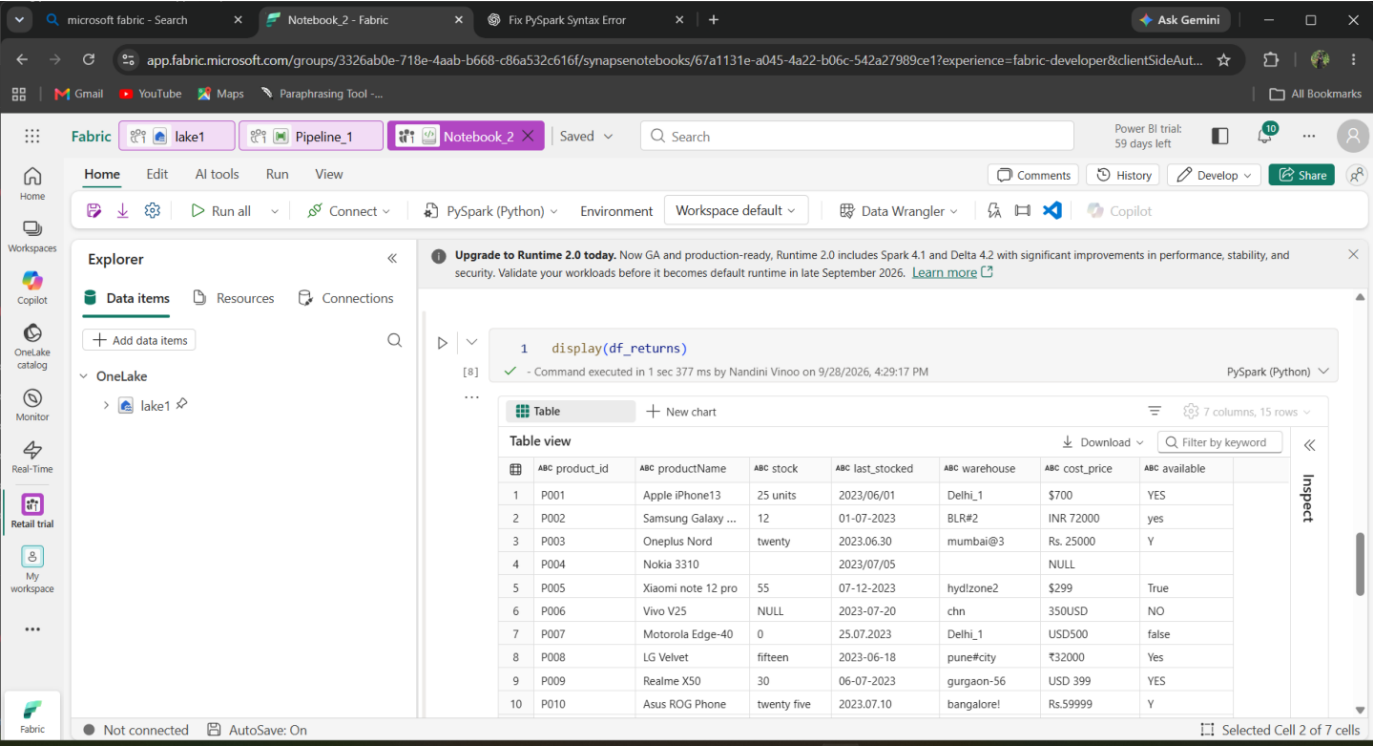


Figure 5. A cell labeled `display(df_returns)` shows inventory-shaped data. Source: first PDF, page 5.

Visible data issues

The table includes stock values such as NULL, 0, fifteen and twenty five. The date field uses slash, dot and hyphen separators. Warehouse strings contain symbols and spelling variations. Availability includes YES, yes, Y, True, NO and false.

Important evidence discrepancy

Although the notebook cell calls `display(df_returns)`, the displayed columns are inventory columns. This may indicate a variable assignment mix-up or a copied display cell. The screenshot should not be treated as a validated returns preview. Check that `df_returns` is read from the returns file and has the expected return schema.

Proposed treatment

Apply explicit number-word mapping for stock, controlled availability mapping and date parsing. Keep unresolved values flagged rather than silently converting all blanks to zero. Retain currency information separately from the numeric amount.

8. Raw orders profiling

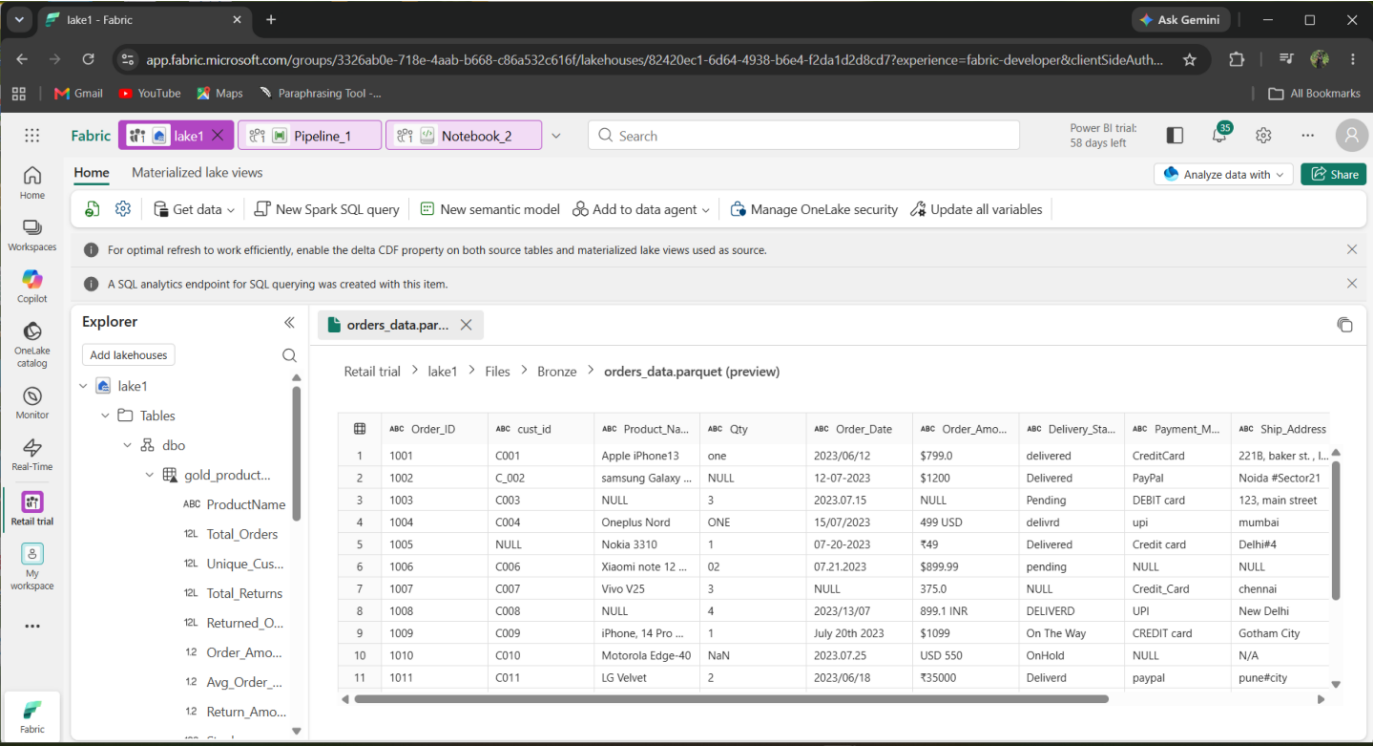


Figure 6. Bronze orders preview alongside the Gold table explorer. Source: first PDF, page 7.

Orders schema

The visible raw columns include Order_ID, cust_id, Product_Name, Qty, Order_Date, Order_Amount, Delivery_Status, Payment_Method and Ship_Address; some names are truncated on screen. Records include missing product or customer values and inconsistent product-name capitalization.

Examples requiring cleaning

Qty contains one, ONE, 02, NULL and NaN. Dates include 2023/06/12, 12-07-2023, 2023.07.15 and July 20th 2023. Amounts combine numeric text with symbols or codes such as USD and INR. Delivery and payment categories also vary in case and spelling.

Recommended decision rules

Define accepted date formats and reject impossible dates. Normalize known product and customer identifiers. Convert quantities only when the meaning is clear. Mixed currencies require a common reporting currency or separate totals by currency; removing symbols alone is insufficient.

9. Silver table organization

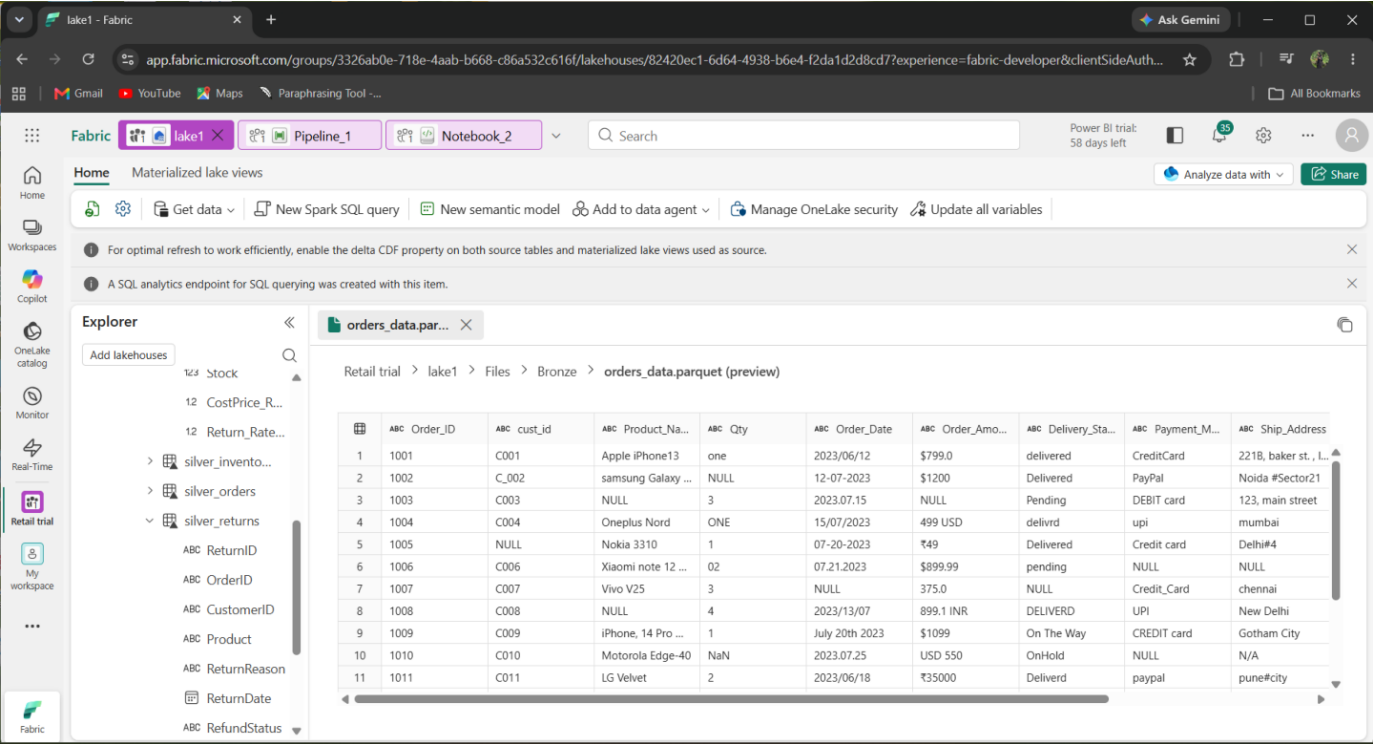


Figure 7. silver_orders, silver_returns and inventory table appear in the explorer. Source: first PDF, page 8.

Observed Silver objects

The explorer shows **silver_orders**, **silver_returns** and a truncated silver inventory name. The expanded returns table lists ReturnID, OrderID, CustomerID, Product, ReturnReason, ReturnDate and RefundStatus. A later screenshot shows the full inventory table name as silver_inventory.

Role of Silver

Separate tables allow each dataset to be typed and validated before joining. Orders provide the transaction base, returns reference related orders, and inventory provides product stock and cost attributes. ReturnDate is displayed with a date icon, providing evidence of date typing for that field.

Evidence boundary

The open grid is still the Bronze orders file, not the Silver returns table. The explorer confirms that the tables exist but does not reveal every cleaning rule or prove that all quality checks have passed.

10. Enriching orders with related data

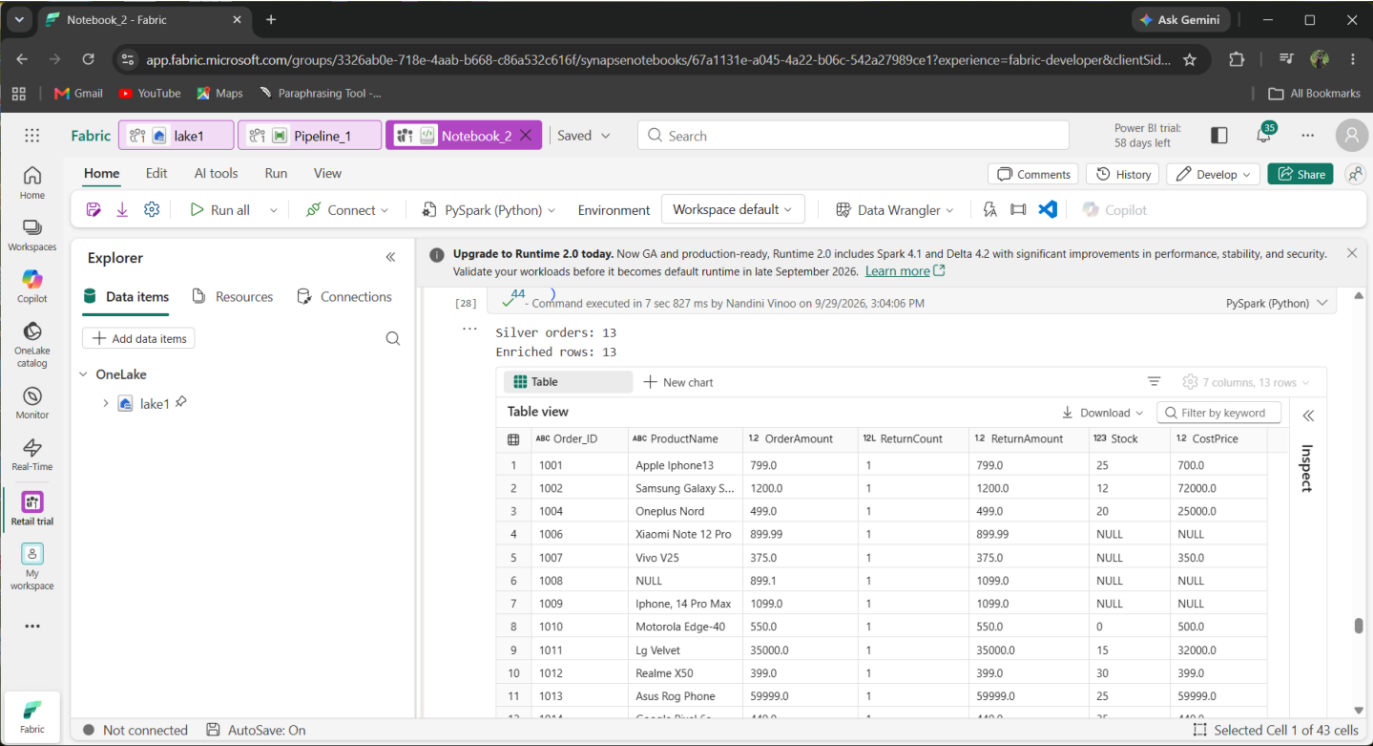


Figure 8. Enriched notebook output with 13 rows. Source: first PDF, page 6.

Observed result

The notebook prints **Silver orders: 13** and **Enriched rows: 13**. The grid contains Order_ID, ProductName, OrderAmount, ReturnCount, ReturnAmount, Stock and CostPrice. It combines transaction values with return and inventory attributes.

What the count comparison means

The displayed enrichment did not change the row count from 13. This is a useful check against obvious join multiplication, but it does not prove the join keys were unique or every match was correct. The full join expressions are not visible.

Remaining quality issues

Some enriched ProductName, Stock and CostPrice values remain NULL. Order 1008 shows OrderAmount 899.1 and ReturnAmount 1099.0. This difference needs reconciliation to the source or business rule; it should not be assumed to be a valid refund. The reason raw orders reduce to 13 is not demonstrated.

11. Gold product KPI fields

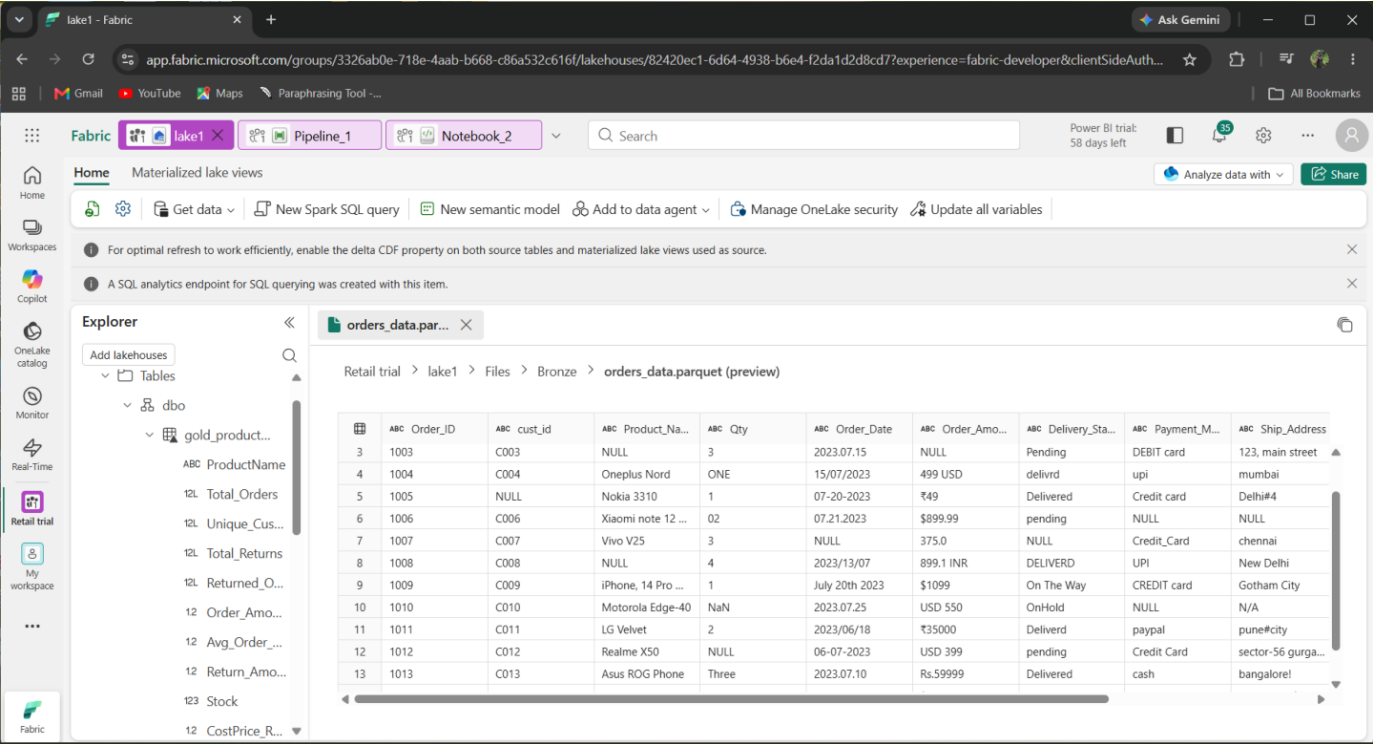


Figure 9. Gold table field list in the lakehouse explorer. Source: first PDF, page 9.

Analytical table

The explorer shows a Gold product table, and the semantic-model screenshot later identifies its full name as **gold_product_kpis**. Fields include ProductName, Total_Orders, Unique_Customers, Total_Returns, Returned_Orders, Order_Amount_Raw_Sum, Avg_Order_Amount, Return_Amount_Raw_Sum, Stock, CostPrice_Raw and Return_Rate_Pct; some are truncated in this figure.

Intended grain

The reporting fields suggest a product-level summary: one analytical group per standardized product name. Confirm this by checking uniqueness of ProductName in the actual Gold table. A product-group count is not directly evidenced by the 13-row enrichment output.

Why this layer helps

Preparing reporting fields in Gold reduces repeated data preparation in Power BI. Raw_Sum naming also signals that the monetary values require business interpretation before being described as normalized revenue or net profit.

12. Checking source coverage and table presence

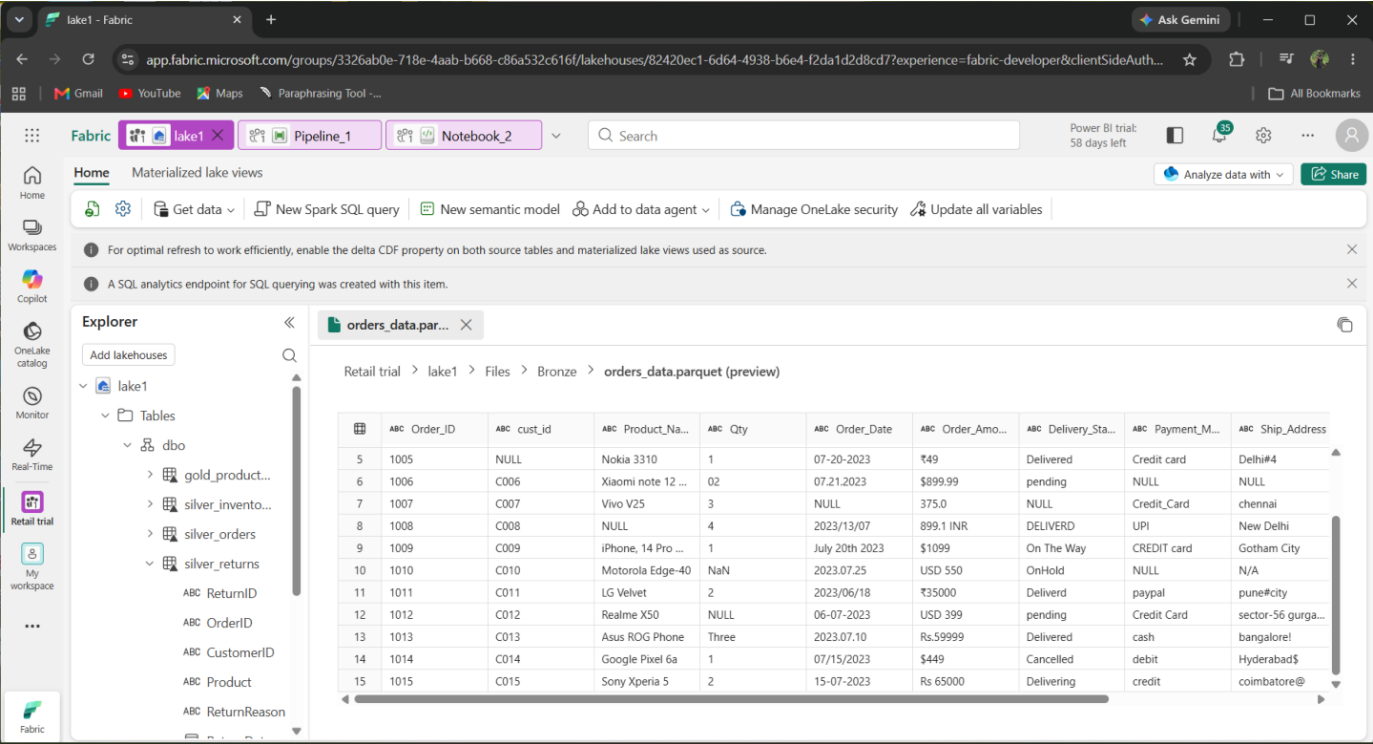


Figure 10. Later raw order records and Silver/Gold table names. Source: first PDF, page 10.

Observed source range

The raw preview includes orders through Order_ID 1015, with earlier screenshots showing Order_ID 1001 onward. The lakehouse explorer contains the Gold table, silver_inventory, silver_orders and silver_returns. This confirms a visible separation between source files and analytical tables.

Reconciliation requirement

Compare raw orders, accepted Silver orders, rejected rows, enriched rows and Gold product groups as distinct counts. The screenshot sequence shows 15 numbered raw entries and a 13-row Silver/enriched result, but does not explain which rules removed records or whether duplicates were consolidated.

Recommended audit output

Record rejection reasons such as invalid date, missing key or duplicate order. A reject table and count summary would make the transformation traceable. These controls are proposed additions, not demonstrated completed features.

13. Creating the semantic model

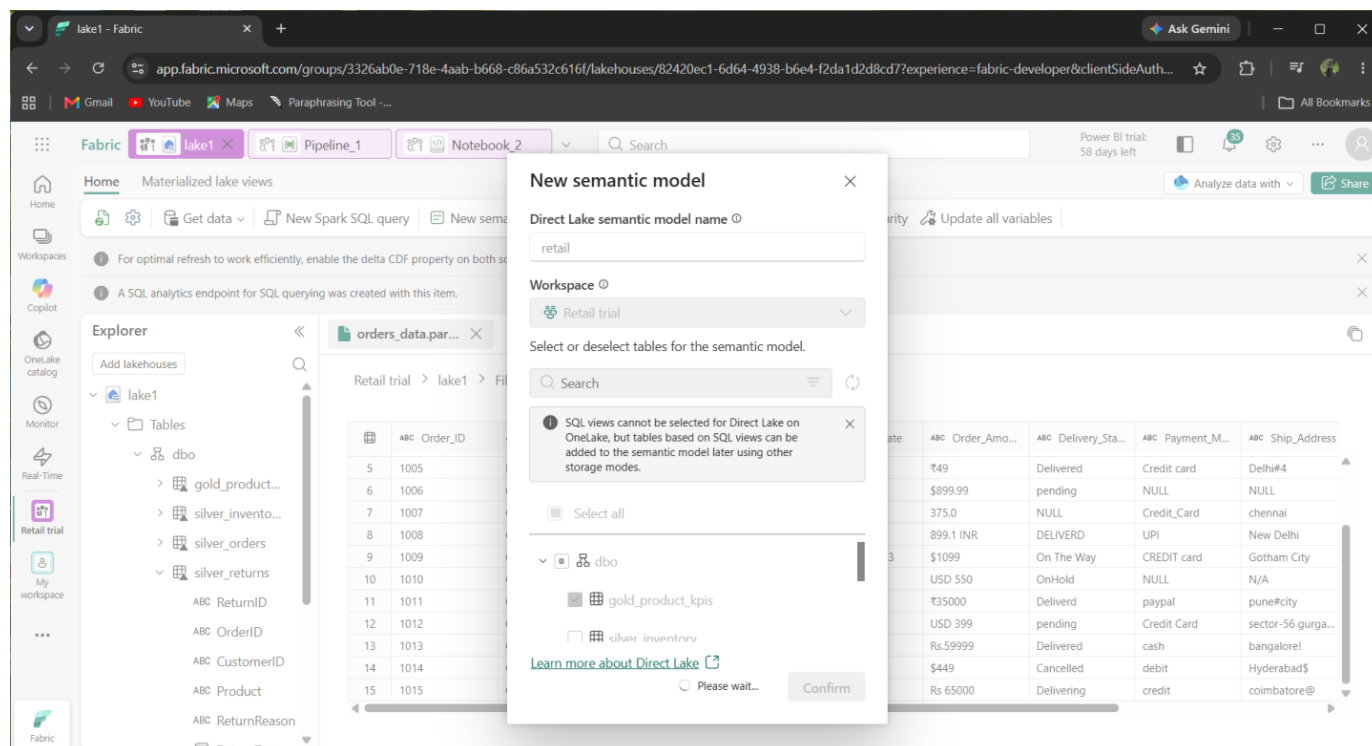


Figure 11. New semantic model dialog with Gold table selection. Source: Part 2 PDF, page 1.

Observed setup

The dialog names the model **retail**, selects workspace **Retail trial** and checks **gold_product_kpis** under **dbo**. It identifies the setup as a Direct Lake semantic model. The next screenshot shows the resulting model with the Gold table.

Purpose

The semantic model makes analytical data available to Power BI through business-facing fields and calculations. Direct Lake is a Fabric semantic-model storage mode [3]. The dialog alone does not confirm every table storage property or a production refresh configuration.

Scope choice

Using only the Gold table gives this demonstration a simple model. The attachments do not show relationships to separate product, customer or date dimensions. Such dimensions could be introduced later when additional analytical questions require them.

14. Model fields and aggregation behavior

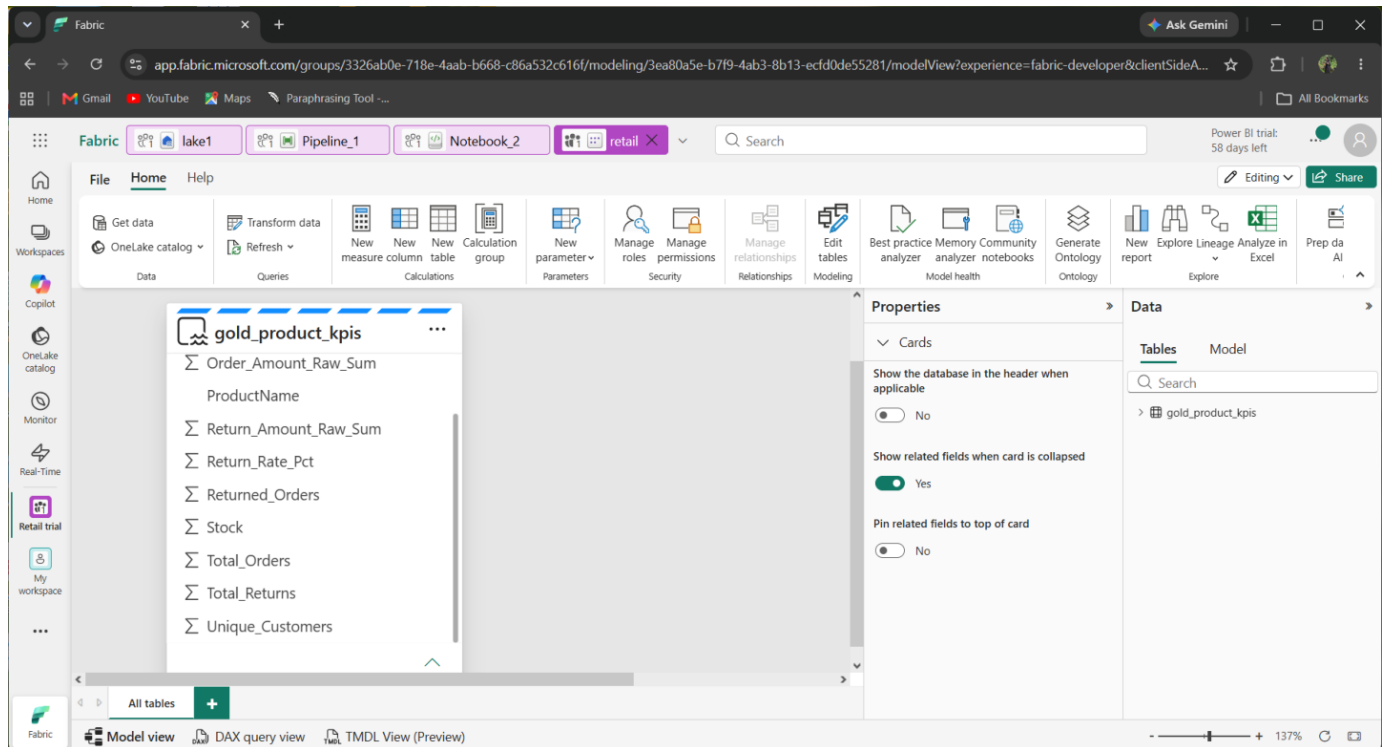


Figure 12. retail model with gold_product_kpis. Source: Part 2 PDF, page 2.

Available report fields

The model view exposes Order_Amount_Raw_Sum, ProductName, Return_Amount_Raw_Sum, Return_Rate_Pct, Returned_Orders, Stock, Total_Orders, Total_Returns and Unique_Customers. The summation icons indicate fields available for numerical aggregation.

Aggregation decisions

Counts and transaction amounts can often be summed at compatible grain. Product return rates and averages require calculated measures for the total context. Distinct customer counts across products can overlap; summing product-level Unique_Customers may count the same customer more than once.

Stock caution

Stock is an inventory snapshot, not an order quantity. Validate that the Gold preparation contributes each product stock value once. Summing inventory repeated on multiple order rows would overstate total stock.

15. Initial Power BI report construction

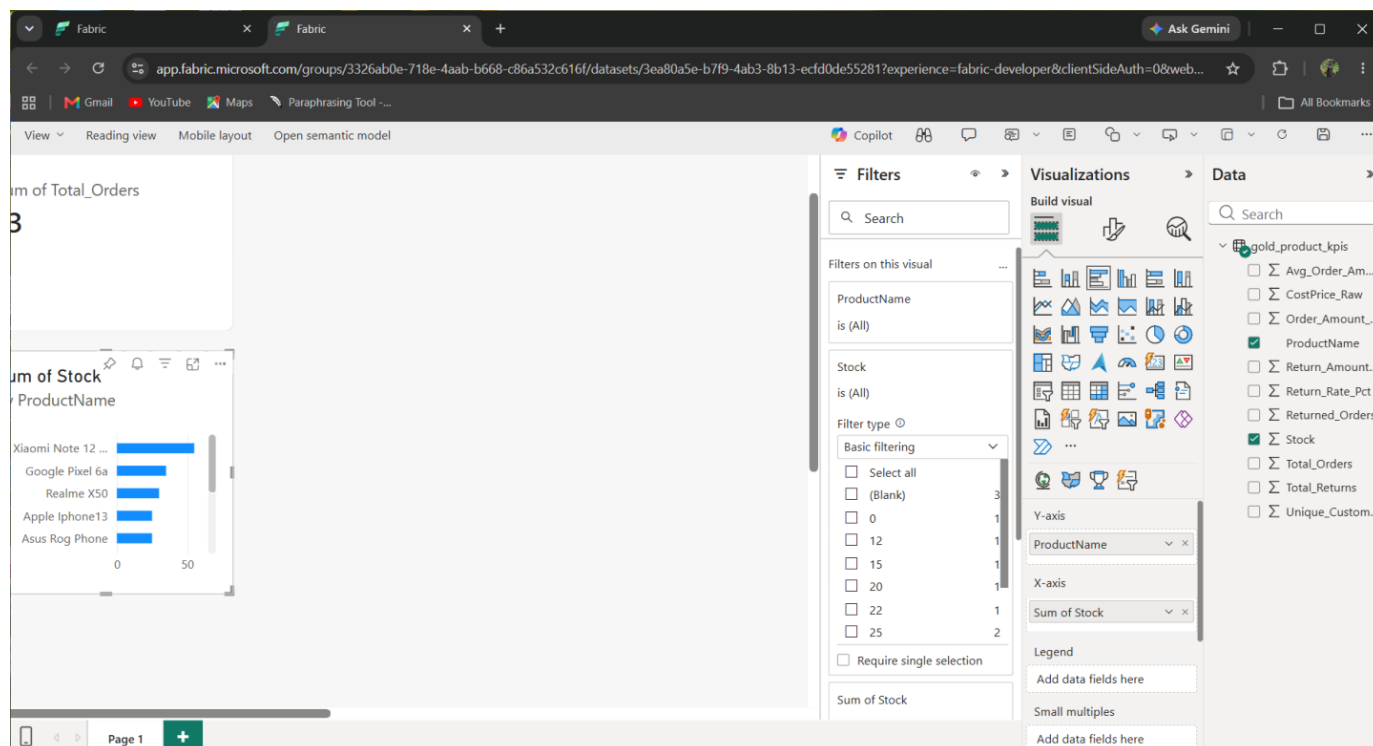


Figure 13. Initial card and stock chart in the report editor. Source: Part 2 PDF, page 3.

Building the first visuals

The screenshot shows a Total Orders card and a horizontal stock chart. The chart uses ProductName on the category axis and Sum of Stock on the value axis. The visual filter list includes a blank stock category, illustrating why null handling matters to presentation.

Filtering and formatting

A visual-level filter can exclude a blank category for display, while a ProductName slicer can filter multiple visuals together. Excluding a blank from one chart does not repair the underlying table or remove the record from all report calculations.

Validation step

Test the report with all products and with one product selected. Card, table and chart values should agree under the same filter. Confirm that filters and interactions are intentionally applied rather than relying on appearance alone.

16. Final Retail Sales Overview report

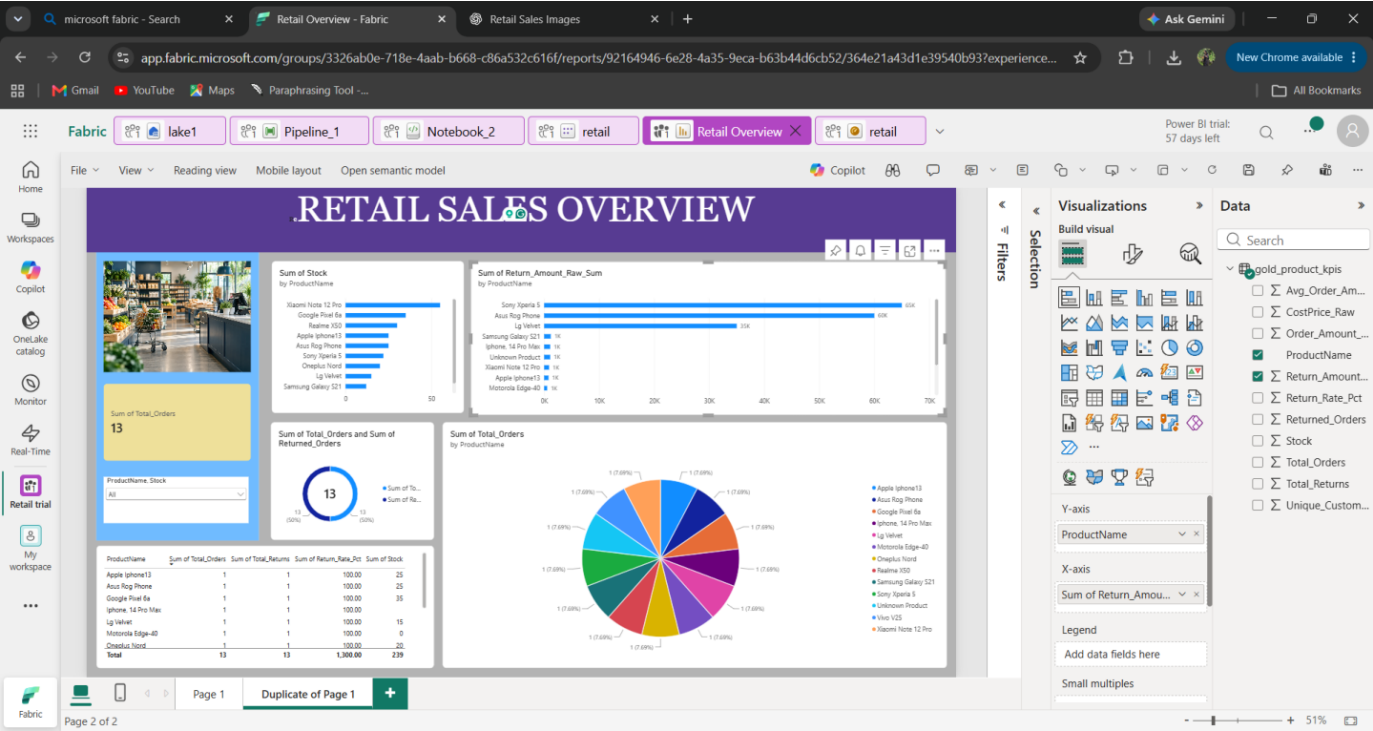


Figure 14. User-supplied final report screenshot. Source: first PDF, page 1.

Visible layout

The page uses a purple title banner, a left sidebar with a retail image, Total Orders card and product dropdown, plus a grid of charts and a summary table. The card displays 13. The table totals show 13 orders, 13 returns and 239 stock.

Same charts, accurate interpretation

The upper-right horizontal chart is **Return_Amount_Raw_Sum by ProductName**, not a sales chart. Sony Xperia 5 has the largest visible return amount, approximately 65K, followed by Asus Rog Phone at approximately 60K. Stock bars show Xiaomi Note 12 Pro with the largest visible stock value.

Report caveats

The pie shows one order per displayed product group, so many slices are equal. The donut compares orders and returned orders, which overlap. Its 50/50 display is not a 50% return rate. The table sums Return_Rate_Pct to 1,300.00; an overall return-rate measure is needed.

17. Business metric definitions

The following definitions are proposed reporting semantics. Validate them against the actual notebook before presenting them as implemented logic. A field name alone does not prove its formula.

Metric	Recommended interpretation
Total Orders	Count accepted distinct Order_ID values for each product; establish rules for cancelled orders.
Total Returns	Count return events or distinct ReturnID values. Several returns may reference one order.
Returned Orders	Count distinct orders with at least one qualifying return.
Return Rate	Returned Orders / Total Orders. Format as a percentage.
Order Amount Raw Sum	Sum parsed source order amounts. Confirm whether these are line totals or unit prices and reconcile currency.
Return Amount Raw Sum	Sum parsed return amounts; reconcile to order amount and currency.
Average Order Amount	Total comparable order amount / Total Orders; recompute for totals.
Unique Customers	Distinct accepted customer identifiers in the filter context.
Stock	Latest applicable product stock snapshot, without order-level duplication.

Example overall return-rate measure

```
Overall Return Rate =  
DIVIDE(  
SUM(gold_product_kpis[Returned_Orders]),  
SUM(gold_product_kpis[Total_Orders])  
)
```

Format the measure as Percentage. This example is valid only if Gold product groups do not duplicate orders across products, Returned_Orders is a distinct qualifying count, and the underlying grain has been validated. For multi-product orders, calculate order-level distinct counts from an appropriate fact table instead.

If 13 qualifying distinct orders are all returned, the overall rate is 100%, not 1,300% and not the donut share of 50%. The screenshots suggest this sample condition but the full return logic is not supplied.

18. Data quality assessment and improvements

Observation	Recommended next action
Mixed currency strings	Retain currency codes. Convert with an approved dated exchange rate or report separate currencies.
Several date formats	Use explicit parsing formats; quarantine unparseable or impossible dates.
Number words in quantities and stock	Map approved words to numbers; flag unknown text.
Missing product values	Use a controlled Unknown Product label where suitable; preserve an unmatched-record flag.
Inventory mismatch after enrichment	Join on stable product identifiers when available; document name normalization rules.
df_returns displays inventory columns	Recheck file path, variable assignment and returns schema before further processing.
Raw 15 entries; Silver result 13	Provide accepted/rejected count reconciliation and reasons.
Order 1008 amount mismatch	Reconcile order and return values to original records.
1,300% total rate	Replace sum of percentages with a ratio measure.
Orders/returns donut overlap	Use returned vs non-returned orders if a mutually exclusive share chart is desired.

Validation priorities

First verify source-to-table mappings and join keys, then reconcile counts and amounts, and only then refine formatting. A visually polished page can still show misleading totals if the metric grain or aggregation is incorrect.

The screenshots show useful progress through the analytical layers, but unresolved nulls, mixed currencies and count reconciliation prevent the report from being treated as a fully validated production solution.

19. Repeatable execution and validation plan

Recommended sequence

- 1. Source check:** confirm all three input files exist and record file names, timestamps and batch identity. Check the expected columns before copying.
- 2. Bronze ingestion:** run Pipeline_1 and inspect the activity output. Confirm each destination file exists and reconcile rows read and written. Green dependency connectors alone are not execution evidence.
- 3. Silver preparation:** run the cleaning notebook from the beginning so variables are assigned consistently. Validate data types, key uniqueness, allowed categories and rejected-row counts. Keep reasons for rejected records.
- 4. Enrichment:** validate return-to-order and inventory-to-product keys. Compare row counts before and after joins. Identify duplicate matches and unmatched records rather than silently dropping them.
- 5. Gold preparation:** confirm the product grain, reconcile order and return totals, prevent duplicate stock aggregation, and verify metric definitions. Check the treatment of unknown product groups.
- 6. Semantic model:** expose Gold fields, format measures and verify aggregation behavior. Confirm the report uses the intended model and that model data is current.
- 7. Report testing:** clear filters and record totals. Select Sony Xperia 5 and compare the card, table and charts. Restore the All selection before saving an overview page. Check that labels describe return amounts and sales amounts accurately.

Proposed production extensions

Add parameterized paths, consistent names, scheduled orchestration, logging, failure notifications, incremental ingestion, role-based access and a documented retention policy. None of these features is confirmed as complete by the supplied evidence.

20. Conclusion and references

The supplied screenshots demonstrate a retail analytics workflow across Azure source storage and Microsoft Fabric. Three file formats are landed as Bronze Parquet files, a notebook supports inspection and enrichment, Silver and Gold tables are visible, and a semantic model provides the fields used in the final Power BI report.

The main learning outcome is the integration of ingestion, data preparation and reporting within a layered lakehouse design. The strongest evidence consists of the source container, Bronze folder, 13-row enriched output, Gold field list, semantic-model view and final report screenshot.

Before using the dashboard for business decisions, reconcile the Silver row reduction, confirm return mapping and currency handling, correct percentage totals and verify inventory aggregation. These improvements strengthen the accuracy of the results while preserving the project structure already demonstrated.

Screenshot provenance

Figures 1-14 reproduce every screenshot from the two PDFs supplied with this request. They are reordered into the project sequence. The original final report is reproduced; the generated design preview from the earlier conversation is not used as implementation evidence.

Source A: Retail_Project_Screenshots(1).pdf - 10 pages.

Source B: Retail_Project_Screenshots_Part2(1).pdf - 4 pages.

Technical references

[1] Microsoft Learn, Implement medallion lakehouse architecture in Fabric.

learn.microsoft.com/en-us/fabric/onelake/onelake-medallion-lakehouse-architecture

[2] Microsoft Learn, How to copy data using copy activity.

learn.microsoft.com/en-us/fabric/data-factory/copy-data-activity

[3] Microsoft Learn, Direct Lake overview.

learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview

References checked on 1 October 2026. Technical references explain the platform concepts; they do not independently verify the implementation in the screenshots.